

BSDA2001P - Introduction to DL and GenAI Project

Project: Smart MCQ Solver Challenge

Student: Abhishek Kumar

Roll No.: 23F2004693

1. Problem Statement and Goal

The goal of this project was to build a model that can solve multiple-choice questions. Each question has five options (A-E). The model has to rank the most likely answers, and the Kaggle score is based on MAP@3. This means the correct answer should appear as high as possible in the top three predictions.

My main goal was to understand the full deep learning workflow: preparing text data, training different models, comparing them, tracking experiments, making a Kaggle submission, and finally deploying one trained model as a small web app.

2. Dataset and Preprocessing

The training data contains a question/prompt, five answer choices (A, B, C, D, and E), and the correct answer. The test data has the same question and options but does not include the answer.

The main preprocessing steps were simple:

- ☐ Convert text values to strings and handle missing values safely.
- ☐ Remove extra spaces and keep the original sentence meaning.
- ☐ Create question-option pairs so that each question is compared with all five choices.
- ☐ Use truncation so very long text does not exceed the model input limit.

I avoided heavy cleaning because punctuation and sentence structure can be useful for understanding the question.

3. Methodology

I tried three different approaches so I could compare a model trained from scratch, a pretrained transformer, and a larger instruction model.

Model	Type	Why I used it
BiLSTM	Custom / from scratch	Simple sequence baseline and easy to understand.
DeBERTa-v3-base	Pretrained transformer	Strong text understanding and direct support for multiple-choice scoring.
Qwen2.5-3B-Instruct	Instruction LLM	To compare a larger language model with the discriminative models.

4. Model Architecture

4.1 BiLSTM

The BiLSTM model was trained from scratch. Text is converted into token IDs, passed through an embedding layer, then through a bidirectional LSTM. The final representation is used to score each answer option. Bidirectional processing helps the model read the sequence in both directions.

4.2 DeBERTa-v3-base

DeBERTa was the main fine-tuned transformer model. For each question, I created five pairs: question+A, question+B, and so on. The model gives one score for each pair, and softmax converts those scores into five probabilities. The option with the highest probability becomes the first prediction.

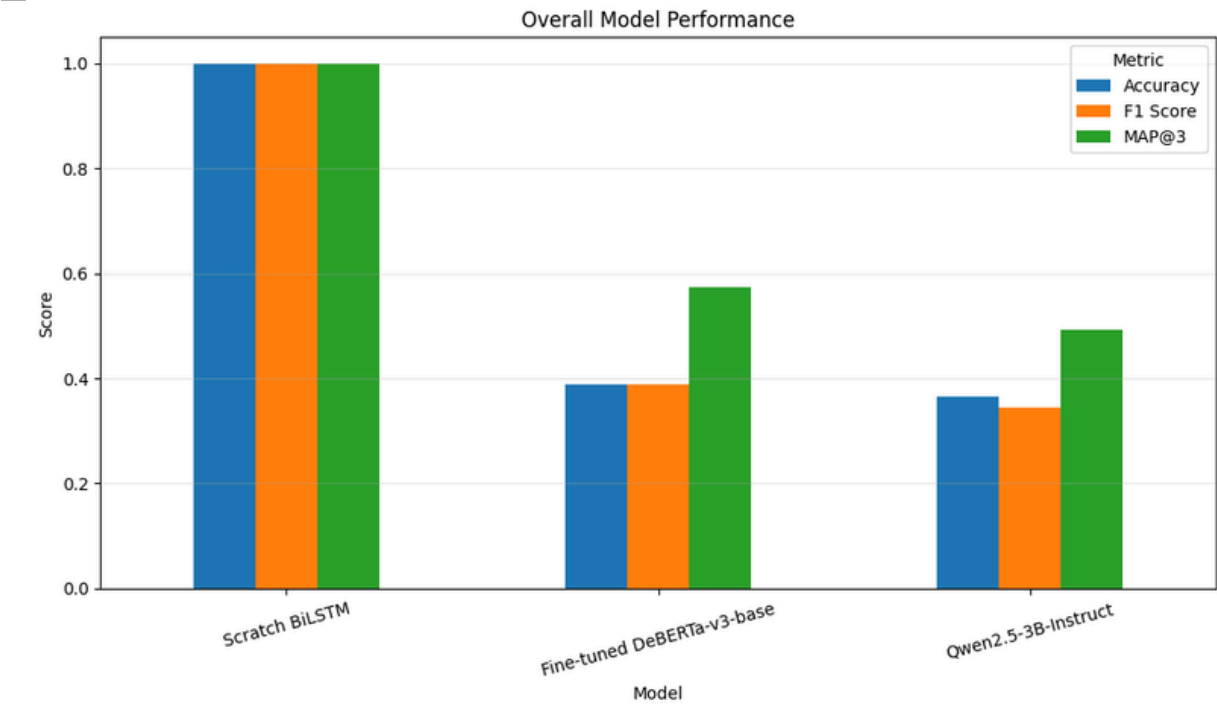
4.3 Qwen2.5-3B-Instruct

Qwen was used as an additional language-model experiment. I used the 3B version because larger versions caused GPU memory and bitsandbytes compatibility problems on Kaggle. This model was much heavier than BiLSTM and DeBERTa, so it was useful mainly for comparison.

5. Evaluation and Results

I evaluated the models using Accuracy, Macro-F1 and MAP@3. The values below are taken directly from the executed notebook.

Model	Accuracy	Macro-F1	MAP@3
Scratch BiLSTM	1.0000	1.0000	1.0000
DeBERTa-v3-base	0.3881	0.3889	0.5732
Qwen2.5-3B	0.3650	0.3440	0.4925



The BiLSTM score of 1.0 on the validation set is unusually high, so I do not assume it will generalize perfectly. The Kaggle leaderboard is a better test of unseen performance. The notebook also searched ensemble weights using trained-model probabilities only. The best validation MAP@3 was 0.99717 with weights 0.40 BiLSTM, 0.10 DeBERTa, and 0.50 Qwen. My final public Kaggle score was about 0.75394, which is above the required 0.73 cutoff. The exact leaderboard rank is not stored in the notebook, so I have not added a made-up rank.

5.1 Experiment Tracking with WandB

The notebook successfully connects to Weights & Biases (WandB). It creates runs for the BiLSTM, DeBERTa, Qwen, and the three-model comparison. The BiLSTM run logs epoch and training loss, DeBERTa is tracked through the Hugging Face Trainer, Qwen logs evaluation metrics, and the comparison run logs the model-comparison table. The notebook confirms the tracking code and successful connection, but it does not contain screenshots from the WandB website.

6. Tokenization and Training Details

For DeBERTa, I used the tokenizer that comes with microsoft/deberta-v3-base through the Hugging Face AutoTokenizer. The question was repeated five times and paired with the five options. Padding and truncation were applied automatically, with a maximum length of 256 tokens.

6.1 DeBERTa Training Setup

Setting	Value
Pretrained model	Learning rate
Learning rate	2e-5
Epochs	4
Train batch size	2
Gradient accumulation	8
Weight decay	0.01
Warmup ratio	0.1
Optimizer	AdamW
Precision	FP32 (FP16 disabled due to gradient error)
Model selection	Best validation MAP@3 checkpoint

During training, I faced a few environmental issues. For example, FP16 training produced an 'unscale FP16 gradients' error, so I used float32. I also had Kaggle session restarts where variables such as tr_df, val_df, or deberta_trainer were lost. I fixed these by making the notebook order clearer and by saving checkpoints after training.

6.2 Experiment Tracking

Weights & Biases (WandB) was used for experiment tracking when the API key was available. The goal was to compare common metrics such as accuracy, macro-F1, and MAP@3 across different runs. For deployment-only reruns, WandB was disabled so that missing login details did not stop training.

7. Evaluation

The main competition metric was MAP@3. I also used accuracy and macro-F1 during validation. MAP@3 is useful here because it gives the highest reward when the correct option is ranked first, a smaller reward when it is second or third, and zero if it is outside the top three.

My final Kaggle public leaderboard score was about 0.75394, which was above the required 0.73 cutoff. The final submission used trained-model probabilities only.

Model	Main role	Speed /cost	Observation
BiLSTM	From-scratch baseline	Low	Simple, fast and useful for learning.
DeBERTa	Fine-tuned MCQ model	Medium	Best choice for deployment because it directly scores options.
Qwen 3B	Additional LLM experiment	High	More compute and slower inference.

8. Deployment

I deployed the DeBERTa model on Hugging Face Spaces using Gradio. The app accepts one question and five options, Then it shows the top three answers with probabilities.

The deployment package contains app.py, config.json, model.safetensors, tokenizer files, requirements.txt and README.md. Because my account could not switch to CPU Basic without a paid plan, I used ZeroGPU. For ZeroGPU, the prediction function was decorated with spaces.GPU and the app successfully loaded the 738 MB model.

9. Error Analysis: Almost Equal 20% Predictions

After deployment, I tested a question whose correct answer was B. The app returned almost equal probabilities: D=20.04%, B=20.01% and E=20.00%.

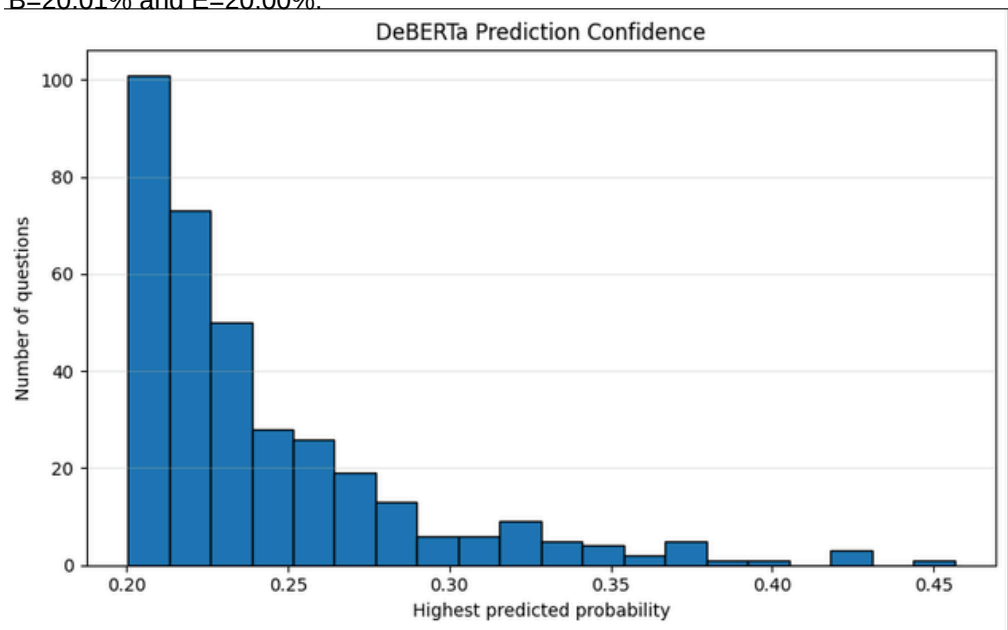


Figure : DeBERTa maximum prediction confidence on validation data.

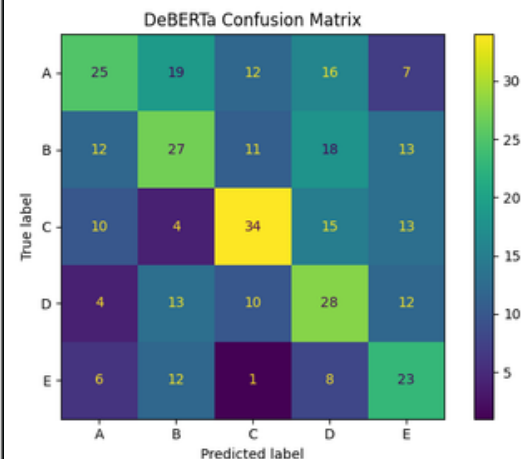


Figure: DeBERTa confusion matrix on the validation split.

With five options, a completely uncertain model gives about 20% to each option. So the bigger problem is not only that D was above B; it is that all five scores are almost the same.

The notebook supports this observation. DeBERTa has validation accuracy 0.3881 and MAP@3 0.5732, and the confidence plot shows many predictions close to 0.20- 0.25. This means the deployed behavior is consistent with a weak or uncertain DeBERTa model.

9.1 Likely Reason

The most likely reason is that the model file used for deployment was not the best fine-tuned checkpoint, or the classifier head did not learn enough during training. The Hugging Face logs showed that all weights loaded successfully, so the web app itself was running. This points more towards a checkpoint/model-quality issue than a Gradio issue.

Another possibility is that the exported checkpoint came from an early or weak training state. When logits for all five options are very close, softmax naturally turns them into probabilities near 0.20.

9.2 How I Would Fix It

- ☐ Load every saved DeBERTa checkpoint in Kaggle.
- ☐ Evaluate each checkpoint on the same validation set and compare MAP@3.
- ☐ Manually test a few known questions before exporting.
- ☐ Choose the checkpoint with the best validation result.
- ☐ Export that exact model and matching tokenizer using `save_pretrained()`.
- ☐ Replace the old model files in Hugging Face and test the same questions again.
- ☐ If all checkpoints still give around 20%, retrain DeBERTa and monitor loss and validation MAP@3.

This was a useful lesson: a deployment can be technically 'running' while the model inside it is still not giving good predictions. Model validation must be done before and after export.

10. Challenges Faced

- ☐ FP16gradient error while training DeBERTa.
- ☐ Kaggle sessions restarted and cleared Python variables.
- ☐ bitsandbytes and CUDA compatibility problems with larger Qwen models.
- ☐ Hugging Face dependency conflicts between Gradio and Transformers.
- ☐ ZeroGPU needed a different setup from a normal CPU Space.
- ☐ The deployed model loaded correctly but produced almost uniform probabilities.

I solved these problems one by one instead of changing the whole project at once. For example, I switched DeBERTa to float32, reduced Qwen to 3B, kept deployment dependencies simple, and used the ZeroGPU decorator for inference.

11. Use of AI Assistance

I used AI tools as a helper during this project. I mainly used them to understand error messages, debug code, check notebook order, improve documentation, and solve deployment issues. For example, AI assistance helped me identify missing imports, missing variables after Kaggle restarts, dependency conflicts, and Hugging Face ZeroGPU setup problems.

I did not treat AI output as automatically correct. I ran the code in my own Kaggle/Hugging Face environment, checked the output, and changed the solution when it did not work. The dataset experiments, training runs, Kaggle submissions, and final deployment were executed in my project environment.

12. Main Insights

- A pretrained transformer is much easier to adapt to an MCQ task than training a large language model from scratch. MAP@3 cares about ranking, not only the first prediction.
- A model checkpoint should always be tested before deployment.
- Almost equal 20% probabilities are a strong sign that the classifier is uncertain.
- Environment and dependency issues are a real part of the ML workflow.
- Smaller models can be more practical when GPU memory is limited.

13. Conclusion and Future Work

This project gave me practical experience with the complete deep learning workflow: data preparation, custom sequence models, transformer fine-tuning, experiment tracking, Kaggle evaluation and deployment.

The final Kaggle score crossed the required cutoff, and the project was deployed as a Gradio app on Hugging Face Spaces.

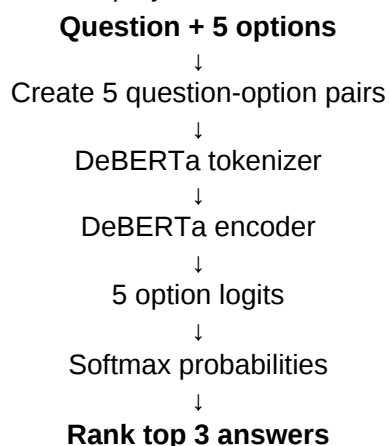
The main remaining issue is the near-uniform probability output from the deployed DeBERTa model. My next step is to verify all saved checkpoints, export the best validated checkpoint, and redeploy it. After that, I would try better hyperparameter tuning and a stronger ensemble if more improvement is needed.

14. References

1. He, P., Liu, X., Gao, J., and Chen, W. DeBERTa: Decoding-enhanced BERT with Disentangled Attention.
2. Microsoft DeBERTa-v3-base model card, Hugging Face.
3. Hugging Face Transformers documentation.
4. PyTorch documentation.
5. Gradio documentation.
6. Hugging Face Spaces / ZeroGPU documentation.
7. Weights & Biases documentation.
8. Kaggle - Smart MCQ Solver Challenge.
9. Qwen2.5 model documentation.

Appendix: Simple Architecture Flow

The main DeBERTa multiple-choice flow used in this project:



Note: This report intentionally uses simple language and short explanations so that the technical decisions are easy to understand and explain during viva.

- **Use of LLM Assistance:**
- I used Large Language Models (LLMs) as an assistant during this project. They were mainly used to understand error messages, debug code, review the notebook flow, and troubleshoot deployment issues.
- For example, I used LLM assistance while fixing missing imports and variables, dependency conflicts, Kaggle runtime issues, and Hugging Face deployment errors.
- I tested the suggested changes in my own environment and verified the outputs before using them.
- The model training, experiments, evaluation, Kaggle submissions, and deployment were carried out through my project environment.